Lab4

Structuring geometric classes with strict naming constraints

Circle

```
- radius : double
+ Circle(radius : double)
+ setRadius(radius : double)
+ getRadius() : double
```

Rectangle

```
- width : double
- height : double
+ Rectangle(width : double, height : double)
+ setWidth(width : double)
+ setHeight(height : double)
+ getWidth() : double
+ getHeight() : double
```

Triangle

```
- base : double
- height : double
+ Triangle(base : double, height : double)
+ setBase(base : double)
+ setHeight(height : double)
+ getBase() : double
+ getHeight() : double
```

Square

```
- width : double
- height : double
+ Square(width : double, height : double)
+ setWidth(width : double)
+ setHeight(height : double)
+ getWidth() : double
+ getHeight() : double
```

Architectural Rules:

1. All constructors and setter methods strictly utilize parameter names identical to their corresponding class properties.
2. Every class explicitly requires standard constructor, getter, and setter operations.

- **Requirements:**
  1. Implement previous classes
  2. Make sure you are using list initializer in contractors.
  3. For class circle, make the setter function regular, and getter function inline.
  4. In main, define pointer to object for the all classes.
  5. Call setting and getter function from the created pointers.

- <u>Output questions, what is the out put of the following:</u>

```
class Student{
private:
    int id;
    string name;
public:
    Student (int id, string name) {
     this.id=id;
     this.name=name;
     cout<<this.id<<":"<<this.name;
     }
};
int main (){
 Student s1(1,"Ali").
}
```

Output

```
class Circle{
private:
    double radius;
public:
    void setRadius(double r){
     radius=r; }
    double getRadius()const{
     return radius; }
};
int main(){
 Circle c1;
 c1.radius=5.5;
 cout<<getRadius();
}
```

Output

- Write simple code for the following:

You have a **Distance** class that has **feet** as int, and **inches** as float, Based on this:

2. Define an overloaded constructor that sets **ft** parameter to feet and **inc** param to inches. (Initialization Must be in the same line using **initializer list** not in the body of constructor).

3. Define an overloaded constructor (with **default values** 1 and 1) that set **feet** parameter to feet attribute and **inches** param to inches attribute. (initialization Must be in the body of constructor).